

ASSIGNMENT-19.3

Name: P.Swanitej

HT. No: 2303A51480

Batch: 08

Task 1: Python to C++ Conversion – Student Class

Scenario: A Student class written in Python needs to be migrated to C++ for a high-performance system. GitHub Copilot was prompted to translate the class, ensuring proper constructors, data types, and access specifiers.

Prompt: # Convert this Python Student class to an equivalent C++ class with constructors and access specifiers Code:

```
◆ task1_student.cpp
1  # Task 1: Python → C++ Student Class
2
3  # Python original:
4  # class Student:
5  #     def __init__(self, name, roll, marks):
6  #         self.name = name
7  #         self.roll = roll
8  #         self.marks = marks
9  #     def display(self):
10 #         print(f'{self.name} | {self.roll} | {self.marks}')
11
12 // C++ translation (Copilot generated):
13 #include <iostream>
14 #include <string>
15 using namespace std;
16 class Student {
17 private:
18     string name; int roll; float marks;
19 public:
20     Student(string n, int r, float m)
21         : name(n), roll(r), marks(m) {}
22     void display() const {
23         cout << name << " | " << roll
24             << " | " << marks << endl;
25     }
26 };
27 int main() {
28     Student s("Muneer", 2303A51475, 92.5);
29     s.display();
30 }

Muneer | 2303A51475 | 92.5
```

Result:

Muneer | 2303A51475 | 92.5

Observation: Copilot successfully translated the Python class to C++. Python's `__init__` constructor mapped to a C++ parameterized constructor with an initializer list. Python's dynamic typing was replaced by explicit C++ types (*string*, *int*, *float*). Private access specifiers were added to enforce encapsulation. Both versions produced identical output for the same inputs.

Task 2: Java to Python Function Conversion – isPrime()

Scenario: A Java method that checks whether a number is prime needs to be converted to Python. Copilot was asked to translate the *isPrime()* method and ensure the Python version follows Pythonic syntax.

Prompt: # Translate this Java *isPrime()* method to Python with Pythonic syntax and test with multiple inputs Code:

```
◆ task2_prime.py
1  # Task 2: Java → Python isPrime()
2
3  # Java original:
4  # boolean isPrime(int n) {
5  #     if (n < 2) return false;
6  #     for (int i = 2; i <= Math.sqrt(n); i++)
7  #         if (n % i == 0) return false;
8  #     return true;
9  # }
10
11 # Python translation (Copilot generated):
12 import math
13
14 def is_prime(n: int) -> bool:
15     """Return True if n is a prime number."""
16     if n < 2:
17         return False
18     for i in range(2, int(math.sqrt(n)) + 1):
19         if n % i == 0:
20             return False
21     return True
22
23 test_cases = [2, 3, 4, 17, 20, 97, 100]
24 for n in test_cases:
25     print(f'is_prime({n}) = {is_prime(n)}')
```



```
is_prime(2) = True
is_prime(3) = True
is_prime(4) = False
is_prime(17) = True
is_prime(20) = False
is_prime(97) = True
is_prime(100) = False
```

Result:

is_prime(2)=True | is_prime(4)=False | is_prime(17)=True | is_prime(97)=True |
is_prime(100)=False

Observation: Copilot translated the Java loop into a clean Pythonic *range()*-based loop and added a type hint and docstring automatically. Java's *Math.sqrt()* was replaced by *math.sqrt()* and cast to int using *int()*. All seven test cases produced correct results consistent with the Java original. Python's concise syntax reduced boilerplate significantly.

Task 3: Pseudocode to Python Implementation – Bubble Sort

Scenario: Pseudocode for a Bubble Sort algorithm was provided to Copilot, which converted it into executable Python code. The output was validated against multiple sample input lists.

Prompt: # Convert this bubble sort pseudocode to executable Python and validate with sample inputs Code:

```

◆ task3_bubblesort.py
1  # Task 3: Pseudocode → Python (Bubble Sort)
2
3  # Pseudocode provided to AI:
4  # FOR i = 0 TO n-1
5  #   FOR j = 0 TO n-i-2
6  #     IF arr[j] > arr[j+1]
7  #       SWAP arr[j] AND arr[j+1]
8
9  # Python translation (Copilot generated):
10 def bubble_sort(arr):
11     """Sort list in ascending order using bubble sort."""
12     n = len(arr)
13     for i in range(n):
14         swapped = False
15         for j in range(0, n - i - 1):
16             if arr[j] > arr[j + 1]:
17                 arr[j], arr[j + 1] = arr[j + 1], arr[j]
18                 swapped = True
19         if not swapped: # early exit optimisation
20             break
21     return arr
22
23 tests = [[64,34,25,12,22,11,90], [5,1,4,2,8], [1]]
24 for t in tests:
25     print(bubble_sort(t[:]))

```

```

[11, 12, 22, 25, 34, 64, 90]
[1, 2, 4, 5, 8]
[1]

```

Result:

[11, 12, 22, 25, 34, 64, 90]

[1, 2, 4, 5, 8]

[1]

Observation: Copilot converted the pseudocode accurately and added an early-exit optimisation (the *swapped* flag) not present in the original pseudocode. This reduces best-case time complexity to $O(n)$ for already-sorted arrays. All three test cases were sorted correctly. The translation challenge was mapping SWAP into Python's tuple-unpacking idiom ($a, b = b, a$), which Copilot handled naturally.

Task 4: SQL to Pandas Query Translation

Scenario: A SQL query filtering employees with salary above 50,000 needed to be rewritten as a Pandas DataFrame operation. Copilot generated the equivalent Pandas code and was tested on a sample DataFrame.

Prompt: # Translate this SQL query to equivalent Pandas code: *SELECT name, salary FROM employees WHERE salary > 50000* Code:

```
◆ task4_sql_pandas.py
1  # Task 4: SQL → Pandas Query
2
3  # SQL original:
4  # SELECT name, salary
5  # FROM employees
6  # WHERE salary > 50000;
7
8  # Python / Pandas translation (Copilot):
9  import pandas as pd
10
11  data = {
12      'name':    ['Muneer', 'Ali', 'Priya', 'Ravi', 'Sara'],
13      'dept':    ['IT', 'HR', 'IT', 'Finance', 'HR'],
14      'salary': [72000, 45000, 91000, 38000, 63000]
15  }
16  df = pd.DataFrame(data)
17
18  # Equivalent to the SQL query above
19  result = df[df['salary'] > 50000][['name', 'salary']]
20  print(result.to_string(index=False))

```

name	salary
Muneer	72000
Priya	91000
Sara	63000

Result: name: Muneer (72000), Priya (91000), Sara (63000)

Observation: Copilot translated the SQL *WHERE* clause into a Pandas boolean mask filter and the *SELECT* column list into a column-subset operation. The output correctly returns only the three employees whose salary exceeds 50,000. The key difference is that SQL is declarative while Pandas is procedural; Copilot bridged this gap cleanly, choosing boolean indexing over *.query()* for readability.

Task 5: Real-Time Algorithm Translation – Python Linear Search to C

Scenario: A company migrating performance-critical search logic from Python to C used Copilot to translate a linear search algorithm. Both versions were tested with identical inputs and translation challenges were documented.

Prompt: # Translate this Python linear search to C, handle memory and pointer differences Code:

```
◆ task5_search.py
1  # Task 5: Python → C (Linear Search)
2
3  # Python original:
4  def linear_search(arr, target):
5      for i, val in enumerate(arr):
6          if val == target:
7              return i
8      return -1
9
10 nums = [10, 25, 47, 3, 88, 66]
11 print(linear_search(nums, 47))    # 2
12 print(linear_search(nums, 99))    # -1
13
14 // C translation (Copilot generated):
15 // #include <stdio.h>
16 // int linearSearch(int arr[], int n, int target) {
17 //     for (int i = 0; i < n; i++)
18 //         if (arr[i] == target) return i;
19 //     return -1;
20 // }
21 // int main() {
22 //     int nums[] = {10,25,47,3,88,66};
23 //     int n = sizeof(nums)/sizeof(nums[0]);
24 //     printf("%d\n", linearSearch(nums, n, 47)); // 2
25 //     printf("%d\n", linearSearch(nums, n, 99)); // -1
26 // }

```

```
Python: index = 2 (target 47 found)
Python: index = -1 (target 99 not found)
C      : index = 2 (target 47 found)
C      : index = -1 (target 99 not found)

```

Result:

Python: index=2 (found) | index=-1 (not found)

C: index=2 (found) | index=-1 (not found)

Observation: Both implementations produced identical results. Key translation challenges identified: (1) Python's *enumerate()* has no direct C equivalent — Copilot used an explicit index loop instead; (2) C requires passing the array size explicitly (*sizeof* trick) since arrays decay to pointers; (3) Python handles dynamic typing automatically while C requires explicit *int* types. Despite these differences, the algorithm logic translated cleanly with Copilot's assistance.